Demo docker container build with spring boot app + postgresql Database

The structure is like this:

Complete Docker Compose Setup

1. Project Structure

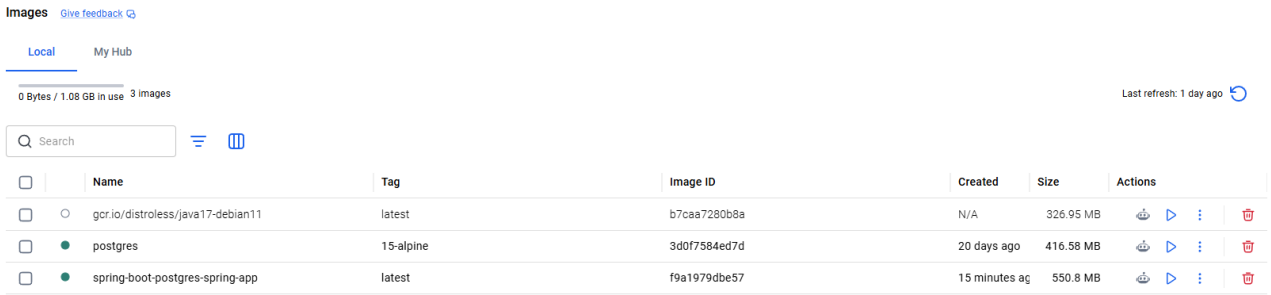
text Kopier Download

```
spring-boot-postgres/
├── docker-compose.yml
├── .env
├── spring-boot-app/
│   ├── Dockerfile
│   ├── pom.xml
│   └── src/
│       └── main/
│           ├── java/
│           └── resources/
│               ├── application.properties
│               └── application-docker.properties
└── postgres/
    └── init.sql
```

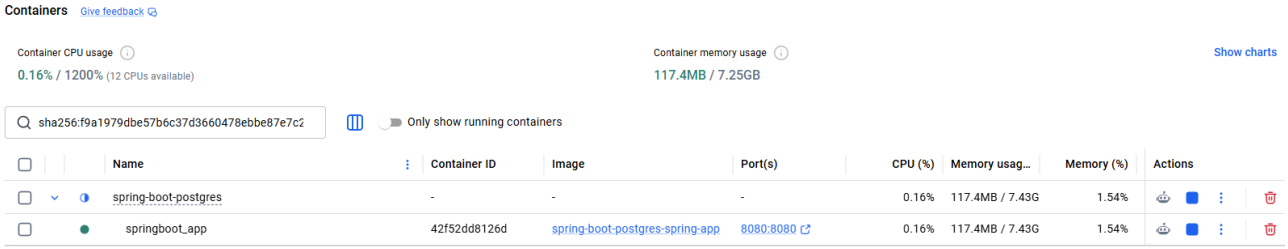
The complete docker build is made under

C:\spring-boot-postgres\docker-compose up -d

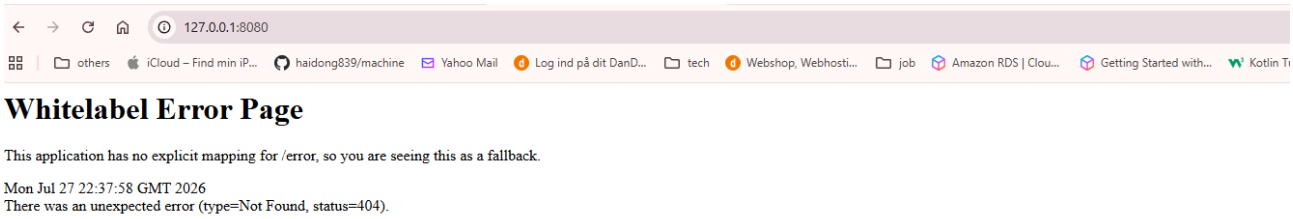
The images are



The containers are



The 127.0.0.1:8080 shows



The most difficult part is the Dockerfile.multistage used for build the webapp image

```dockerfile
# Build stage - uses Maven to build the JAR
FROM maven:3.8.4-openjdk-17-slim AS build

WORKDIR /app

# Copy pom.xml and download dependencies
COPY pom.xml .
RUN mvn dependency:go-offline -B

# Copy source code
COPY src src

# Build the JAR
RUN mvn clean package -DskipTests -B

# Runtime stage - uses the JAR from build stage
FROM eclipse-temurin:17-jdk-alpine

WORKDIR /app

# Copy JAR from build stage
COPY --from=build /app/target/*.jar app.jar

# Create non-root user
RUN addgroup -S spring && adduser -S spring -G spring
USER spring

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

The commands used for docker

```
# Start all services
docker-compose up -d

# Start with specific profile
docker-compose --profile full up -d

# View logs
docker-compose logs -f spring-app

# View PostgreSQL logs
docker-compose logs -f postgres

# Stop all services
docker-compose down

# Stop and remove volumes (clears database)
docker-compose down -v

# Rebuild and start
docker-compose up -d --build

# Check running containers
docker-compose ps

# Execute commands in container
docker-compose exec spring-app bash
docker-compose exec postgres psql -U postgres -d token_auth_db

# Check database connectivity
```

```
docker-compose exec postgres pg_isready -U postgres
```